Assignment Number: 4

Title: Finding the Maximum Value of a List

Objective: The objective of this lab experiment is to write a Python program that can find the maximum value within a given list. This involves iterating through the elements of the list and comparing them to determine the maximum value.
Introduction: In programming, it is often necessary to find the maximum value within a list. This could be useful in various scenarios, such as finding the highest score in a game or identifying the largest element in a dataset. By developing a Python program to find the maximum value in a list, we can enhance our understanding of list manipulation and algorithmic problem-solving.

Algorithm:

1. Start by initializing a variable, `max_value`, to the first element of the list.
2. Iterate through each element in the list using a for loop.
3. For each element, compare its value with the current `max_value`.
4. If the element's value is greater than `max_value`, update `max_value` with the new value.
5. Repeat steps 3 and 4 until all elements in the list have been processed.
6. Once the iteration is complete, the final value of `max_value` will represent the maximum value within the list.

Python Code:

```
def find_max_value(lst):
    max_value = lst[0]
    for num in lst:
        if num > max_value:
            max_value = num
    return max_value

# Test the function with the given list
my_list = [2, 34, 5, 65, 6, 54, 3, 3, 3]
max_value = find_max_value(my_list)
print("The maximum value in the list is:", max_value)
```

Result Analysis & Discussion: To validate the algorithm, we tested it with the given list [2, 34, 5, 65, 6, 54, 3, 3, 3]. The algorithm correctly identified the maximum value as 65, which is the largest element in the list. This demonstrates the effectiveness of the algorithm in finding the maximum value within a given list.

Conclusion: In conclusion, this lab experiment focused on developing a Python program to find the maximum value within a given list. By iterating through the elements of the list and comparing them, the algorithm successfully identified the maximum value. The results obtained from testing the algorithm with the provided list confirmed its accuracy and efficiency. This exercise enhanced our understanding of list manipulation and algorithmic problem-solving techniques, providing a valuable learning experience.

Assignment Number: 5

Title: Finding the Maximum Value of a Set

Objective: The objective of this lab experiment is to write a Python program that can find the maximum value within a given set. This involves iterating through the elements of the set and comparing them to determine the maximum value.
Introduction: In programming, it is often necessary to find the maximum value within a set. This could be useful in various scenarios, such as finding the highest temperature in a dataset or identifying the largest element in a collection. By developing a Python program to find the maximum value in a set, we can enhance our understanding of set manipulation and algorithmic problem-solving.

Algorithm:

1. Start by initializing a variable, `max_value`, to the first element of the set.
2. Iterate through each element in the set using a for loop.
3. For each element, compare its value with the current `max_value`.
4. If the element's value is greater than `max_value`, update `max_value` with the new value.
5. Repeat steps 3 and 4 until all elements in the set have been processed.
6. Once the iteration is complete, the final value of `max_value` will represent the maximum value within the set.

Python Code:

```
def find_max_value(my_set):
    max_value = next(iter(my_set))
    for num in my_set:
        if num > max_value:
            max_value = num
    return max_value

# Test the function with the given set
```

```
my_set = {2, 34, 5, 65, 6, 54, 3, 3, 3}
max_value = find_max_value(my_set)
print("The maximum value in the set is:", max_value)
```

Result Analysis & Discussion: To validate the algorithm, we tested it with the given set {2, 34, 5, 65, 6, 54, 3, 3, 3}. The algorithm correctly identified the maximum value as 65, which is the largest element in the set. This demonstrates the effectiveness of the algorithm in finding the maximum value within a given set.

Conclusion: In conclusion, this lab experiment focused on developing a Python program to find the maximum value within a given set. By iterating through the elements of the set and comparing them, the algorithm successfully identified the maximum value. The results obtained from testing the algorithm with the provided set confirmed its accuracy and efficiency. This exercise enhanced our understanding of set manipulation and algorithmic problem-solving techniques, providing a valuable learning experience.

Assignment Number: 6

Title: Finding the Maximum Value of a Tuple

Objective: The objective of this lab experiment is to write a Python program that can find the maximum value within a given tuple. This involves iterating through the elements of the tuple and comparing them to determine the maximum value.
Introduction: In programming, it is often necessary to find the maximum value within a tuple. A tuple is an immutable sequence of elements, similar to a list, but with the key difference that it cannot be modified once created. By developing a Python program to find the maximum value in a tuple, we can enhance our understanding of tuple manipulation and algorithmic problem-solving.

Algorithm:

1. Start by initializing a variable, `max_value`, to the first element of the tuple.
2. Iterate through each element in the tuple using a for loop.
3. For each element, compare its value with the current `max_value`.
4. If the element's value is greater than `max_value`, update `max_value` with the new value.
5. Repeat steps 3 and 4 until all elements in the tuple have been processed.

6. Once the iteration is complete, the final value of `max_value` will represent the maximum value within the tuple.

Python Code:

```
def find_max_value(my_tuple):
    max_value = my_tuple[0]
    for num in my_tuple:
        if num > max_value:
            max_value = num
    return max_value

# Test the function with the given tuple
my_tuple = (2, 34, 5, 65, 6, 54, 3, 3, 3)
max_value = find_max_value(my_tuple)
print("The maximum value in the tuple is:", max_value)
```

Result Analysis & Discussion: To validate the algorithm, we tested it with the given tuple (2, 34, 5, 65, 6, 54, 3, 3, 3). The algorithm correctly identified the maximum value as 65, which is the largest element in the tuple. This demonstrates the effectiveness of the algorithm in finding the maximum value within a given tuple.

Conclusion: In conclusion, this lab experiment focused on developing a Python program to find the maximum value within a given tuple. By iterating through the elements of the tuple and comparing them, the algorithm successfully identified the maximum value. The results obtained from testing the algorithm with the provided tuple confirmed its accuracy and efficiency. This exercise enhanced our understanding of tuple manipulation and algorithmic problem-solving techniques, providing a valuable learning experience.

Assignment Number: 7

Title: Age Group Classification Program

Objective: The objective of this lab experiment is to write a Python program that prompts the user to enter their age and prints the corresponding age group based on predefined ranges. The program should accurately classify the user's age into one of the following groups: Child, Teenager, Adult, or Senior Citizen.

Introduction: Age classification is a common task in programming, especially when dealing with user input. By developing a Python program that can determine the age group based on the user's input, we can enhance our understanding of conditional statements and user interaction in Python.

Algorithm:

1. Prompt the user to enter their age.
2. Read the user's input and store it in a variable.
3. Use conditional statements (if-elif-else) to determine the age group based on the following ranges:
    - If the age is between 0 and 12 (inclusive), classify the user as a Child.
    - If the age is between 13 and 19 (inclusive), classify the user as a Teenager.
    - If the age is between 20 and 59 (inclusive), classify the user as an Adult.
    - If the age is 60 or above, classify the user as a Senior Citizen.
4. Print the corresponding age group based on the classification.

Python Code:

```
def classify_age(age):
    if age >= 0 and age <= 12:
        return "Child"
    elif age >= 13 and age <= 19:
        return "Teenager"
    elif age >= 20 and age <= 59:
        return "Adult"
    else:
        return "Senior Citizen"

# Prompt the user to enter their age
age = int(input("Enter your age: "))

# Classify the age and print the corresponding age group
age_group = classify_age(age)
print("Your age group is:", age_group)
```

Result Analysis & Discussion: To validate the algorithm, we tested it with different age inputs. The program correctly classified the age into the respective age groups based on the predefined ranges. For example, if the user entered an age of 25, the program correctly classified them as an Adult. Similarly, if the user entered an age of 70, the program classified them as a Senior Citizen. This demonstrates the accuracy of the algorithm in determining the age group based on the user's input.

Conclusion: In conclusion, this lab experiment focused on developing a Python program to classify the user's age into different age groups based on predefined ranges. By using conditional statements, the program accurately determined the age group corresponding to the user's input. The results obtained from testing the program with various age inputs confirmed its accuracy and effectiveness. This exercise enhanced our

understanding of conditional statements and user interaction in Python, providing a valuable learning experience.

Assignment Number: 8

Title: Body Mass Index (BMI) Calculation Program

Objective: The objective of this lab experiment is to write a Python program that prompts the user to enter their weight (in kilograms) and height (in meters) and calculates their Body Mass Index (BMI) using the formula: BMI = weight / (height * height). The program should then classify the BMI into one of the following categories: Underweight, Normal weight, Overweight, or Obesity.
Introduction: The Body Mass Index (BMI) is a widely used measure to assess whether an individual's weight is within a healthy range for their height. By developing a Python program that calculates the BMI and classifies it into different categories, we can gain a better understanding of how to use mathematical formulas and conditional statements in Python.

Algorithm:

1. Prompt the user to enter their weight in kilograms.
2. Read the user's input and store it in a variable.
3. Prompt the user to enter their height in meters.
4. Read the user's input and store it in a variable.
5. Calculate the BMI using the formula: BMI = weight / (height * height).
6. Use conditional statements (if-elif-else) to classify the BMI into the following categories:
    - If the BMI is less than 18.5, classify the user as Underweight.
    - If the BMI is between 18.5 and 24.9 (inclusive), classify the user as Normal weight.
    - If the BMI is between 25 and 29.9 (inclusive), classify the user as Overweight.
    - If the BMI is 30 or greater, classify the user as Obesity.
7. Print the calculated BMI and the corresponding category.

Python Code:

```
def calculate_bmi(weight, height):
    bmi = weight / (height * height)
    return bmi
```

```
# Prompt the user to enter their weight in kilograms
weight = float(input("Enter your weight in kilograms: "))

# Prompt the user to enter their height in meters
height = float(input("Enter your height in meters: "))

# Calculate the BMI
bmi = calculate_bmi(weight, height)

# Classify the BMI and print the corresponding category
if bmi < 18.5:
    category = "Underweight"
elif bmi >= 18.5 and bmi <= 24.9:
    category = "Normal weight"
elif bmi >= 25 and bmi <= 29.9:
    category = "Overweight"
else:
    category = "Obesity"

# Print the calculated BMI and the category
print("Your BMI is:", bmi)
print("Category:", category)
```

Result Analysis & Discussion: To validate the algorithm, we tested the program with different weight and height inputs. The program correctly calculated the BMI using the provided formula and classified it into the respective categories based on the predefined ranges. For example, if the user entered a weight of 70 kilograms and a height of 1.8 meters, the program correctly calculated the BMI as 21.6 and classified it as Normal weight. Similarly, if the user entered a weight of 90 kilograms and a height of 1.7 meters, the program calculated the BMI as 31.1 and classified it as Obesity. This demonstrates the accuracy of the algorithm in calculating the BMI and classifying it into the appropriate category.

Conclusion: In conclusion, this lab experiment focused on developing a Python program to calculate the Body Mass Index (BMI) based on the user's weight and height inputs. By using the BMI formula and conditional statements, the program accurately calculated the BMI and classified it into different categories: Underweight, Normal weight, Overweight, or Obesity. The results obtained from testing the program with various weight and height inputs confirmed its accuracy and effectiveness. This exercise enhanced our understanding of mathematical formulas, conditional statements, and user interaction in Python, providing a valuable learning experience.

Assignment number : 9

Title: Lab Report – Function's operation

Objective:

The objective of this lab report is to explore different types of functions in Python and understand their implementation. We will focus on four types of functions:

1. No parameter, no return value
2. Has parameter and return value
3. Has parameter, no return value
4. No parameter, return value

Introduction:

Functions are a fundamental concept in programming that allow us to break down complex tasks into smaller, reusable blocks of code. They help improve code organization, readability, and reusability. In this lab report, we will explore four different types of functions and their implementation in Python.

Algorithm:

1. No parameter, no return value:
    - Define a function named "find_sum_by_list" without any parameters.
    - Inside the function, create a list of numbers.
    - Calculate the sum of the numbers in the list using the built-in `sum()` function.
    - Print the sum.
2. Has parameter and return value:
    - Define a function named "substring_occurrence" that takes two parameters: a string and a substring.
    - Inside the function, use the `count()` method to count the number of occurrences of the substring in the string.
    - Return the count.
3. Has parameter, no return value:
    - Define a function named "find_factorial" that takes one parameter: an integer.
    - Inside the function, initialize a variable "factorial" with a value of 1.
    - Use a loop to calculate the factorial of the given integer.
    - Print the factorial.

4. No parameter, return value:
    - Define a function named "find_power" without any parameters.
    - Inside the function, prompt the user to enter a base number and an exponent.
    - Calculate the power value using the `**` operator.
    - Return the power value.

Python code:

```
# No parameter, no return value
def find_sum_by_list():
    numbers = [2, 34, 5, 65, 6, 54, 3, 3, 3]
    total_sum = sum(numbers)
    print("The sum of the numbers is:", total_sum)

# Has parameter and return value
def substring_occurrence(string, substring):
    count = string.count(substring)
    return count

# Has parameter, no return value
def find_factorial(number):
    factorial = 1
    for i in range(1, number + 1):
        factorial *= i
    print("The factorial of", number, "is:", factorial)

# No parameter, return value
def find_power():
    base = int(input("Enter the base number: "))
    exponent = int(input("Enter the exponent: "))
    power = base ** exponent
    return power

# Main program
find_sum_by_list()

string = "This is a test string. This string has multiple
occurrences of the word string."
substring = "string"
occurrences = substring_occurrence(string, substring)
print("The substring", substring, "occurs", occurrences, "times
in the string.")

number = 5
find_factorial(number)
```

```
power_value = find_power()
print("The power value is:", power_value)
```

Result analysis & discussion:

- The `find_sum_by_list()` function successfully calculates the sum of the numbers in the given list and prints the result.
- The `substring_occurrence()` function counts the number of occurrences of a substring in a given string and returns the count. In our example, it correctly counts the occurrences of the word "string" in the test string.
- The `find_factorial()` function calculates the factorial of a given number and prints the result. In our example, it correctly calculates the factorial of 5.
- The `find_power()` function prompts the user to enter a base number and an exponent, calculates the power value, and returns the result. The returned power value is then printed.

Conclusion:

In this lab report, we explored different types of functions in Python. We implemented functions with no parameters and no return values, functions with parameters and return values, functions with parameters and no return values, and functions with no parameters and return values. We successfully executed the functions and analyzed the results.

Assignment Number: 10

Title:

File Operation: Read, Write, Append, and Close Modes in Python

Objective:

The objective of this lab report is to understand the various file operation modes in Python, including read, write, append, and close modes. We will explore the usage of these modes and their corresponding functions to perform file operations.

Introduction:

File operations are an essential part of any programming language, including Python. Python provides built-in functions and modes to read, write, append,

and close files. Understanding these modes and their usage is crucial for effective file handling in Python.

Algorithm:

1. Open the file in the desired mode (read, write, append) using the `open()` function.
2. Perform the required file operation using the appropriate functions.
3. Close the file using the `close()` function to release system resources.

Python Code:

```
# 1. Read Mode
def read_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(content)
    except IOError:
        print("Error: Could not read the file.")

# 2. Write Mode
def write_file(filename, text):
    try:
        with open(filename, 'w') as file:
            file.write(text)
            print("File written successfully.")
    except IOError:
        print("Error: Could not write to the file.")

# 3. Append Mode
def append_file(filename, text):
    try:
        with open(filename, 'a') as file:
            file.write(text)
            print("Text appended to the file.")
    except IOError:
        print("Error: Could not append to the file.")

# 4. Close Mode
def close_file(filename):
```

```
    try:
        file = open(filename, 'r')
        # Perform file operations
        file.close()
        print("File closed successfully.")
    except IOError:
        print("Error: Could not close the file.")

# Example Usage
filename = "example.txt"
text = "This is a sample text."

# Read Mode
read_file(filename)

# Write Mode
write_file(filename, text)

# Append Mode
append_file(filename, "\nThis is an appended text.")

# Close Mode
close_file(filename)
```

Result Analysis & Discussion:

The provided Python code demonstrates the usage of different file operation modes. Here is a brief analysis of each mode:

1. Read Mode:
    - The `read_file()` function reads the contents of the file specified by the `filename` parameter using the 'r' mode.
    - The `open()` function is used to open the file in read mode, and the `read()` function is used to read the file's contents.
    - The content is then printed to the console.
2. Write Mode:
    - The `write_file()` function writes the specified `text` to the file specified by the `filename` parameter using the 'w' mode.

    - The `open()` function is used to open the file in write mode, and the `write()` function is used to write the text to the file.
3. Append Mode:
    - The `append_file()` function appends the specified `text` to the file specified by the `filename` parameter using the 'a' mode.
    - The `open()` function is used to open the file in append mode, and the `write()` function is used to append the text to the file.
4. Close Mode:
    - The `close_file()` function demonstrates the closing of a file after performing file operations.
    - The `open()` function is used to open the file in read mode, and the `close()` function is used to close the file.

Conclusion:

File operations are essential for reading, writing, appending, and closing files in Python. Understanding the different modes and their corresponding functions allows us to perform file handling tasks effectively. By utilizing the provided Python code and the explanations given, you can now perform file operations in Python with ease.

Assignment Number: 11

Title:

Simple Calculator: Addition, Subtraction, Multiplication, and Division in Python

Objective:

The objective of this lab report is to create a simple calculator program in Python that can perform basic arithmetic operations such as addition, subtraction, multiplication, and division. We will explore the implementation of these operations using Python functions and demonstrate their usage.

Introduction:

A calculator is a fundamental tool used for performing mathematical calculations. In this lab, we will develop a simple calculator program in Python that can handle basic arithmetic operations. By utilizing Python's built-in functions and operators, we can create a user-friendly calculator that can perform calculations efficiently.

Algorithm:

1. Prompt the user to enter two numbers and the desired operation.
2. Based on the user's input, perform the corresponding arithmetic operation using Python functions and operators.
3. Display the result to the user.

Python Code:

```
# Addition
def add(num1, num2):
    return num1 + num2

# Subtraction
def subtract(num1, num2):
    return num1 - num2

# Multiplication
def multiply(num1, num2):
    return num1 * num2

# Division
def divide(num1, num2):
    if num2 != 0:
        return num1 / num2
    else:
        return "Error: Division by zero is not allowed."

# User Input
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
operation = input("Enter the operation (+, -, *, /): ")

# Perform Calculation
```

```
if operation == "+":
    result = add(num1, num2)
elif operation == "-":
    result = subtract(num1, num2)
elif operation == "*":
    result = multiply(num1, num2)
elif operation == "/":
    result = divide(num1, num2)
else:
    result = "Error: Invalid operation entered."

# Display Result
print("Result:", result)
```

Result Analysis & Discussion:

The provided Python code demonstrates the implementation of a simple calculator program. Here is a brief analysis of the code:

1. Addition:
    - The `add()` function takes two numbers as input and returns their sum using the `+` operator.
2. Subtraction:
    - The `subtract()` function takes two numbers as input and returns their difference using the `-` operator.
3. Multiplication:
    - The `multiply()` function takes two numbers as input and returns their product using the `*` operator.
4. Division:
    - The `divide()` function takes two numbers as input and returns their quotient using the `/` operator.
    - It also checks for division by zero and handles the error condition appropriately.
5. User Input:
    - The user is prompted to enter two numbers and the desired operation (+, -, *, /).
6. Perform Calculation:
    - Based on the user's input, the corresponding arithmetic operation is performed using the appropriate function.
7. Display Result:

- The result of the calculation is displayed to the user.

Conclusion:

In this lab, we successfully implemented a simple calculator program in Python. By utilizing Python's built-in functions and operators, we were able to perform basic arithmetic operations such as addition, subtraction, multiplication, and division.